

Supplementary Material for “Engineering norm-aware BDI agents”

Michael Winikoff¹, Frank Dignum², Sebastian Rodriguez³, and John Thangarajah³

¹Victoria University of Wellington, michael.winikoff@vuw.ac.nz

²Umeå University, dignum@cs.umu.se

³RMIT University, {sebastian.rodriguez, john.thangarajah}@rmit.edu.au

This document contains supplementary material for the paper “*Engineering norm-aware BDI agents*”. Section 1 provides the source code, and Section 2 gives two additional figures.

Contents

1	Jason code	2
1.1	Code for goal and norms (including action definitions)	2
1.2	Adding the link (1)	3
1.3	Adding Valuing (2)	3
1.4	Defining Preferences (3)	3
1.5	Adding Meta-Reasoning (4)	4
1.6	Code to Test	4
1.7	Modifications	5
1.8	Code to Test the modifications	7
2	Additional Figures	9

1 Jason code

1.1 Code for goal and norms (including action definitions)

```
1 // original plans
2 +!goToWork : distance(near) & best(gtw,p1) ← !walk.
3 +!goToWork : best(gtw,p2) ← !rideShare.
4 +!goToWork : have(car) & best(gtw,p3) ← !drive.
5 +!goToWork : best(gtw,p4) ← !goto(station); !board(train); !wait(arrived); !disembark.
6
7 // Norm plans, first plan: violate norm
8 +!norm1 : passenger & best(upt,p5) ← true.
9 // Second plan: buy ticket to conform with norm
10 +!norm1 : passenger & best(upt,p6) ← !buy_ticket.
11 +!norm1 : (not passenger) & best(upt,p7) ←
12   .print("Norm doesn't apply.").
13
14 // Note: the above 12 lines are included in the paper's body (Fig 4) so don't change the formatting
    or number of lines without also adjusting the paper
15
16 // (see end of file for instructions to run)
17
18 // Example of a "must not" norm
19 +!norm2 : best(n2,pn21) ← .print("Making phone call (IN NORM) ").
20 +!norm2 : best(n2,pn22) ← true. // conform with norm – don't call
21 // Don't have a third plan, because the norm applies to everyone who rides PT
22
23 // Definition of actions
24
25 +!buy_ticket ← .print("Buying ticket"); + have_ticket.
26 +!board(train) ← .print("Boarding train ... checking ticket status ..."); !
    checkTicket; !make_phone_call. // added phone call for Norm2
27 +!goto(station) ← .print("Going to station").
28 +!wait(arrived) ← .print("Waiting on the train").
29 +!disembark ← .print("Disembarking").
30 +!walk ← .print("Walking").
31 +!rideShare : not(rideshare_fail) ← .print("Using rideshare").
32 +!rideShare : rideshare_fail ← .print("Using rideshare - fail!"); ?fail. //force a failure
33 +!drive ← .print("Driving").
34 +!make_phone_call ← true.
35 // Note that making a phone call has to be wrapped in a goal and plan because I don't think Jason
    allows meta-events on actions
36 +!checkTicket : have_ticket ← .print("Boarding train with ticket").
37 +!checkTicket : have_conc_ticket & have_conc_card ← .print("Boarding train with (
    concession) ticket and card").
38 +!checkTicket : have_conc_ticket & not(have_conc_card) ← .print("Boarding train with (
    concession) ticket but no card [VIOL]").
```

```

39 +!checkTicket : not(have_ticket) & not(have_conc_ticket) & (passenger | concession_passenger) ←
40   .print("Boarding train with no ticket [VIOL] ").
41 +!checkTicket : not(have_ticket) & not(have_conc_ticket) & not(passenger) & not(
    concession_passenger) ←
42   .print("Boarding train with no ticket (but not paying passenger) ").

```

1.2 Adding the link (1)

```

1 // The following line implements the link using Jason meta-events
2 // Explanation: when goto(station) is pending, if there is an intention to goToWork, then suspend it,
   run the norm, and then resume.
3 ^!goto(station)[state(pending)] : .intend(goToWork) ← .print(link_activating) ; .suspend(goToWork)
   ; !norm1 ; .resume(goToWork). //ORIG
4 // Hook for norm 2
5 ^!make_phone_call[state(pending)] : .intend(goToWork) ← .print(link_activating) ; .suspend(
   goToWork) ; !norm2 ; .resume(goToWork). //ORIG

```

1.3 Adding Valuing (2)

```

1 v(p1,[speed(slow),cost(0),sust]).
2 v(p2,[speed(fast),cost(3)]).
3 v(p3,[speed(fast),cost(C)]) :- driving_cost(C).
4 v(p4,[speed(medium),sust|V]) :- v(p5,V) | v(p6,V). // Propagated valuing ...
5 v(p5,[reputation(bad),cost(1.5)]).
6 v(p6,[pgcost(1)]).
7 v(p7,[]).
8
9 driving_cost(2).
10
11 // To avoid extending pref with additional attributes we just give these a cost
12 v(pn21,[cost(1)]).
13 v(pn22,[cost(0)]). // to model situation where there is a really good reason to call give this a higher
   cost

```

1.4 Defining Preferences (3)

```

1 // Key attributes: speed, cost (also whether it's a public good), and sustainability
2 // Prefer faster to slower ...
3 // ... with sustainability adjustment
4 pref(A,B) :- A≠B & get_adj_speed(A,SA) & get_adj_speed(B,SB) & SA>SB.
5 // ... and for same speed, prefer lower cost
6 pref(A,B) :- A≠B & get_adj_speed(A,SA) & get_adj_speed(B,SB) & SA==SB & get_cost(A,CA) &
   get_cost(B,CB) & CA<CB.
7
8 // get cost of valuing, including adjusting for value discount
9 get_cost([],0).
10 get_cost([cost(C)|Cs],N) :- get_cost(Cs,N1) & N = N1+C.
11 get_cost([pgcost(C)|Cs],N) :- get_cost(Cs,N1) & value_discount(CA) & N = (CA*C)+N1.
12 get_cost([speed(_)|Cs],N) :- get_cost(Cs,N).

```

```

13 get_cost([sust|Cs],N) :- get_cost(Cs,N).
14 get_cost([reputation(_)|Cs],N) :- get_cost(Cs,N).
15
16 // A number representing the extent to which the agent's values prefer public good expenditure
17 // >1 means negative support, <1 means positive support
18 value_discount(2).
19
20 // Convert sustainability to number, including adjustment for sustainability, if it is valued
21 get_adj_speed(A,N) :- .member(speed(S),A) & (not(.member(sust,A)) | not(value_sustainability)) &
    map_speed(S,N).
22 get_adj_speed(A,N) :- .member(speed(S),A) & .member(sust,A) & value_sustainability &
    map_speed(S,N1) & N=N1+1.
23
24 map_speed(fast,2).
25 map_speed(medium,1).
26 map_speed(slow,0).

```

1.5 Adding Meta-Reasoning (4)

```

1 // (4.1) making the plans and their context conditions visible
2
3 plan(gtw,p1,distance(near)).
4 plan(gtw,p2,true).
5 plan(gtw,p3,have(car)).
6 plan(gtw,p4,true).
7 plan(upt,p5,passenger).
8 plan(upt,p6,passenger).
9 plan(upt,p7,not(passenger)).
10 plan(n2,pn21,true).
11 plan(n2,pn22,true).
12
13 // Jason version – note that this needs some more work to remove "best" from the context condition.
    There needs to be the triggering event, written as (e.g.) {+!goToWork}
14 // jplan(T,L,C) :- .relevant_plan({+!goToWork}, Plan) & Plan =.. [LL,T,C,B] & LL =.. [_,L|_].
15
16
17 // (4.2) defining "best"
18
19 // L1 is best if there isn't an alternative plan L2 for same trigger T with more preferred valuations V2
20 best(G,L1) :- plan(G,L1,C1) & C1 & v(L1,V1) & not( plan(G,L2,C2) & L2 ≠ L1 & C2 & v(L2,V2)
    & pref(V2,V1) ).

```

1.6 Code to Test

```

1 +!go ←
2 // initial beliefs
3 -distance(_); +have(car); +distance(far); +age(18); // +passenger;
4 .print("Case 1 (drive) : "); !goToWork ;

```

```

5  -have(car);
6  .print("Case 2 (rideshare) : "); !goToWork;
7  +value_sustainability; -distance(_); +distance(far); -age(_); +age(4); // -passenger;
8  .print("Case 3 (usePT-NA) : "); !goToWork;
9  -age(_); +age(18); // +passenger;
10 -value_discount(_); +value_discount(2);
11 .print("Case 4 (usePT-Viol) : "); !goToWork;
12 -value_discount(_); +value_discount(0.5);
13 .print("Case 5 (usePT-Conf) : "); !goToWork;
14 -driving_cost(_); +driving_cost(0.5); +have(car);
15 .print("Case 6a: drive despite valuing sust (cheaper driving)"); !
    goToWork;
16 -driving_cost(_); +driving_cost(2); +have(car);
17 .print("Case 6b: driving not cheaper - use PT"); !goToWork;
18 .print("Done. ").

```

1.7 Modifications

```

1  /* Evaluation: demonstrate flexibility by adapting things
2
3  - Change 1: Children exempt, or seniors exempt at certain times
4  - Change 2: Concession: need to have concession card
5  - Change 3: Different ways of getting tickets
6  - Change 4: deciding to work from home, including handling action/plan failure
7
8  */
9
10 // Change 1: Children and seniors (off peak) exempt – also (change 2) define passenger as "full
    paying passenger" (i.e. excluding concession)
11 passenger :- age(N) & N>=5 & (N<65 | peak) & not(concession_passenger).
12
13
14 // Change 2: concession card holders – additional GPT, five cases:
15 // do nothing, get sanctioned
16 +!norm3 : concession_passenger & best(cch,p8) ← true.
17 // buy concession fare, risking sanction for not having card
18 +!norm3 : concession_passenger & not(have_conc_card) & best(cch,p9) ← !buy_concession_ticket(
    no_card).
19 // Buy concession fare (with concession card at hand)
20 +!norm3 : concession_passenger & have_conc_card & best(cch,p10) ← !buy_concession_ticket(
    have_card).
21 // Norm 2 not applicable
22 +!norm3 : not(concession_passenger) & best(cch,p11) ← .print("Norm2 doesn't apply").
23 // Buy a full fare ticket to avoid risk of sanction for not having concession card
24 +!norm3 : concession_passenger & not(have_conc_card) & best(cch,p12) ← !buy_ticket.
25
26 v(p8,[reputation(bad),cost(1.5)]).

```

```

27 v(p9,[pgcost(0.5),cost(N)]) :- conc_sanc(N).
28 v(p10,[pgcost(0.5)]).
29 v(p11,[]).
30 v(p12,[pgcost(1)]).
31
32 plan(cch,p8,concession_passenger).
33 plan(cch,p9,concession_passenger & not(have_conc_card)).
34 plan(cch,p10,concession_passenger & have_conc_card).
35 plan(cch,p11,not(concession_passenger)).
36 plan(cch,p12,concession_passenger & not(have_conc_card)).
37
38 // Anticipated cost (risk “*” cost) of sanction fine for using a concession fare without having
   concession card
39 conc_sanc(1.5).
40
41 +!buy_concession_ticket(X) ← .print("buying concession ticket ",X);+
   have_conc_ticket.
42
43
44 // Change 3: buy tickets at different times
45 // Note: Jason meta-event for state(pending) starts after the plan for the goal has been selected
46 // This caused problems when +!board(train) checked directly for ticket, since it was checking too
   early
47
48 //^!board(train)[state(pending)] : .intend(goToWork) ← .print(link_activating) ; .suspend(goToWork)
   ; !norm1 ; !norm3 ; .resume(goToWork). //CHANGES
49
50
51 // Change 4: deciding to work from home – this turns out to be too simple to be interesting
52 /*
53 This can be modelled in a number of ways. For illustration, we model as a parent goal work, with
   two options: go to office and work from home. Factors that can affect the choice might include a
   norm for (certain) meetings to be face-to-face, or for working at least 3 days a week in the
   office.
54 Since these norms don’t result in any additional actions, and the “comply vs. violate” option is
   already there in the option to work from home, the code below doesn’t need a norm GPT, but
   only valuing and preferences.
55 So, it’s basically adding a valuing attribute that acknowledges some sort of (reputational?) penalty
   for violating the norm, and then having the preference weight that up against the costs (time,
   money, risk of fine/reputation) of going to work.
56 A more interesting cases is what happens if things *fail*, and alternatives need to be tried. This has
   been implemented and works as expected: we try to (in this case) rideshare, but it fails, and we
   are then running late, so despite the reputational damage (and therefore preference) to not
   WFH, we choose to WFH.
57 */
58 +!work : not(too_much_wfh) & best(w,p13) ← .print("Working from home").
59 +!work : too_much_wfh & best(w,p14) ← .print("Working from home (too much)").

```

```

60 +!work : best(w,p15) ← .print("Going to office ..."); !goToWork.
61 // If going to work fails, then try again, but note running late
62 -!work ← +running_late ; !work.
63
64 plan(w,p13,not(too_much_wfh)).
65 plan(w,p14,too_much_wfh).
66 plan(w,p15,true).
67
68 v(p13,[wfh]).
69 v(p14,[wfh,too_much_wfh_reputation]).
70 v(p15,V) :- v(p1,V) | v(p2,V) | v(p3,V) | v(p4,V).
71
72 // In general prefer to WFH, but not if too much
73 pref(V1,V2) :- .member(wfh,V1) & not(.member(too_much_wfh_reputation,V1)).
74 // Prefer to avoid WFH too much, unless running late (for meeting)
75 pref(V1,V2) :- .member(too_much_wfh_reputation,V2) & not(running_late).

```

1.8 Code to Test the modifications

```

1 +!go_new ←
2 // NEW Change 4
3 +distance(far) ; +age(30) ;
4 .print("New: case 1 ok WFH");
5 !work ;
6 +too_much_wfh ;
7 .print("New: case 2 not ok WFH");
8 !work ;
9 .print("New: case 3 not ok WFH, but fail");
10 +rideshare_fail ;
11 !work ;
12 // Changes 1-3
13 // remove beliefs no longer needed
14 -distance(far) ; -age(30) ; -too_much_wfh ; -rideshare_fail ;
15 // initial beliefs
16 +have(car) ; +distance(near) ; +age(30) ; -have_ticket ; -have_conc_ticket ;
17 .print("Case 1 (drive) : ") ; !goToWork ;
18 -have(car) ; -have_ticket ; -have_conc_ticket ;
19 .print("Case 2 (rideshare) : ") ; !goToWork ;
20 +value_sustainability ; -distance(near) ; +distance(far) ; -age(_) ; +age(4) ; -have_ticket ; -
    have_conc_ticket ;
21 .print("NEW Case 1 (usePT-NA-child) : ") ; !goToWork ;
22 -age(_) ; +age(71) ; -have_ticket ; -have_conc_ticket ;
23 .print("NEW Case 1 (usePT-NA-senior-offpeak) : ") ; !goToWork ;
24 +peak ; -have_ticket ; -have_conc_ticket ;
25 .print("NEW Case 1 (usePT-senior-peak) : ") ; !goToWork ;
26 -age(_) ; +age(30) ; -have_ticket ; -have_conc_ticket ;
27 .print("Case 4 (usePT-Viol) : ") ; !goToWork ;

```

```

28  -value_discount(2); +value_discount(0.5); -have_ticket; -have_conc_ticket;
29  .print("Case 5 (usePT-Conf) : "); !goToWork;
30  +concession_passenger; -have_ticket; -have_conc_ticket;
31  .print("NEW case 2 (not have conc_card) = buy full ticket"); !goToWork
    ;
32  -conc_sanc(_); +conc_sanc(0.25); -have_ticket; -have_conc_ticket; -have_conc_card;
33  .print("NEW case 2 (not have conc_card, reduced sanction) = violate
    norm?"); !goToWork;
34  +have_conc_card; -have_ticket; -have_conc_ticket;
35  .print("NEW case 2 (have conc_card) = buy concession ticket "); !
    goToWork;
36  -value_discount(_); +value_discount(3); -have_ticket; -have_conc_ticket;
37  .print("NEW case 2 (have conc_card) = violate norm "); !goToWork;
38  -have_conc_card; -have_ticket; -have_conc_ticket;
39  .print("NEW case 2 (not have conc_card) = violate norm "); !goToWork;
40  .print("Done. ").
41
42
43  // Run the various cases – use "jason go.mas2j" (GUI) or "jason < go.jcli" (CLI)
44  // Comment out either !go (original) or !gonew (changes)
45  // For gonew need to also adjust code by commenting out the line marked ORIG and uncommenting
    the line marked CHANGES
46  !go.
47  ///!go_new.

```


2 Additional Figures

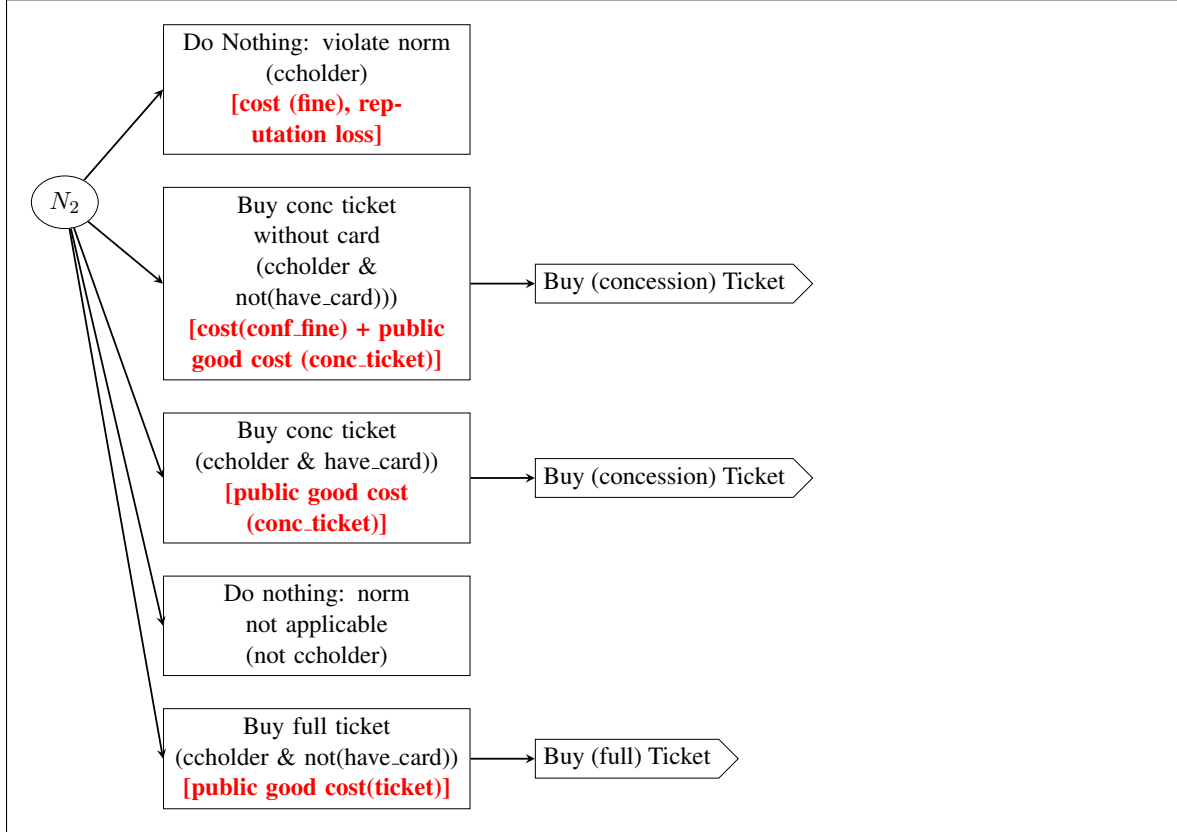


Figure 1: GPT for Norm 2 (G_{N_2})

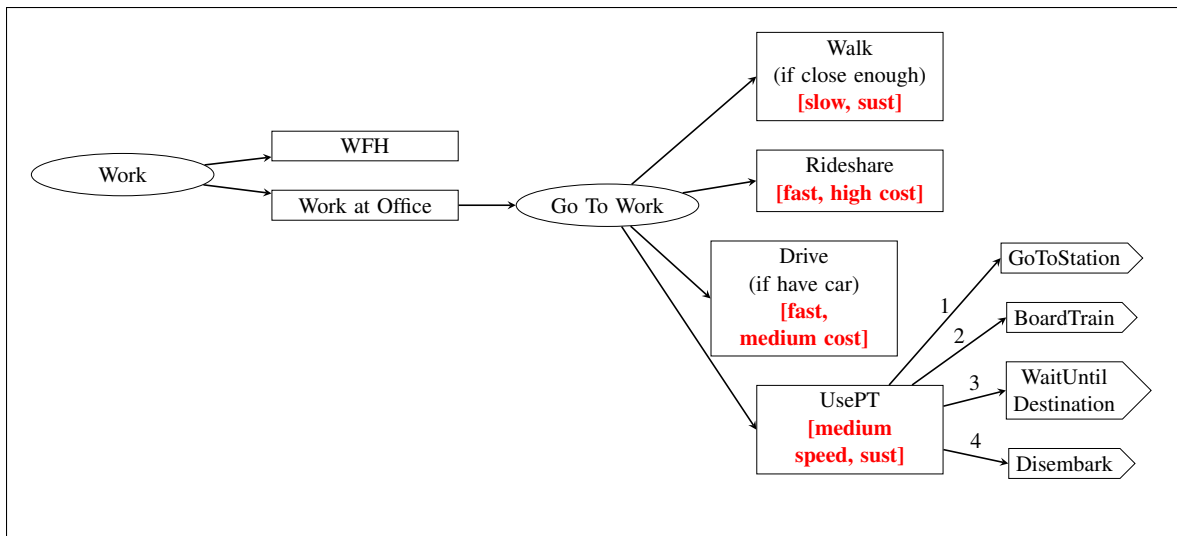


Figure 2: GPT for Change 4